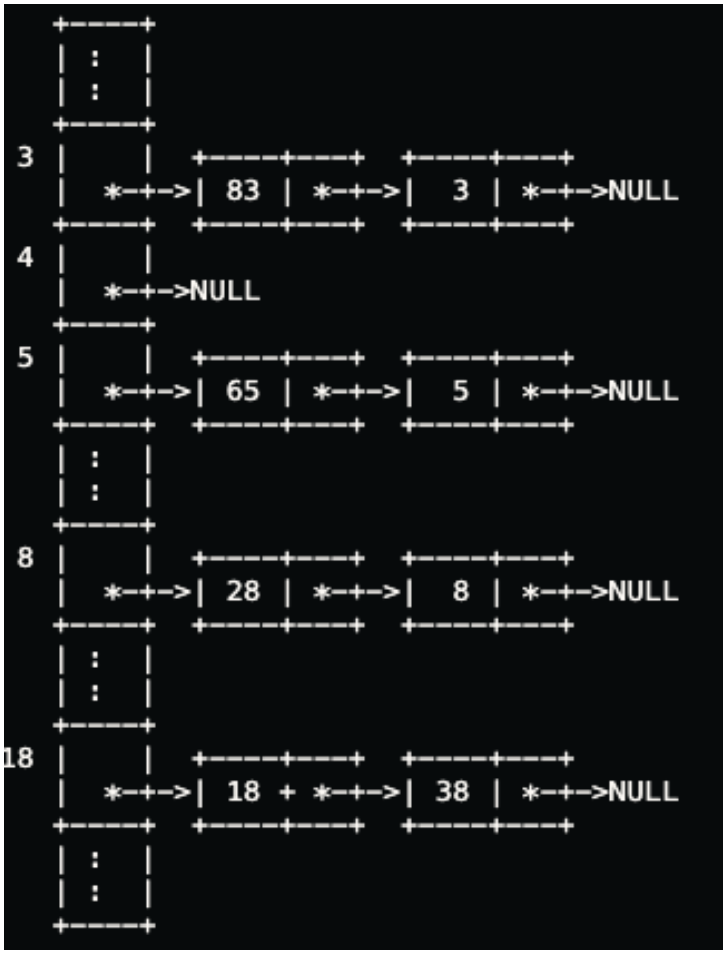
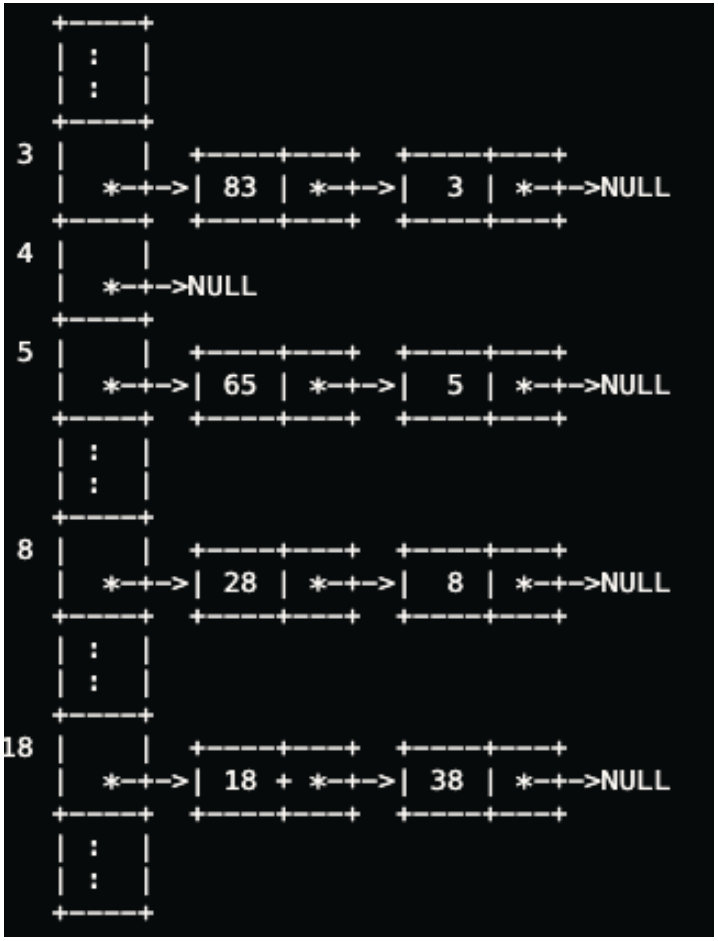
- In the following, say our hash function for n is n % 20.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion?

- In the following, say our hash function for n is n % 20.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion?

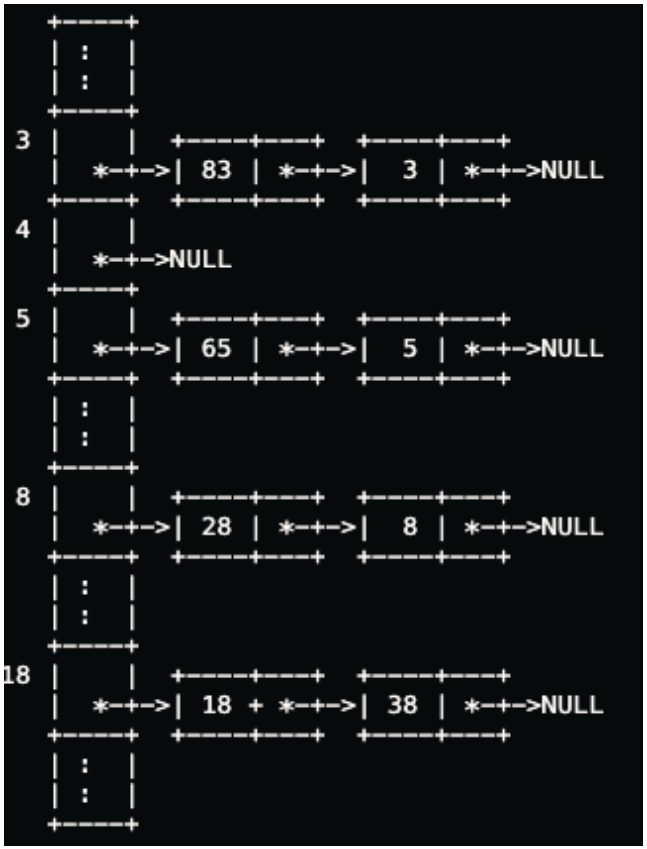


- In the following, say our hash function for n is n % 20.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion?



If we then search for 48, how many of the inserted keys will we look at?

- In the following, say our hash function for n is n % 20 and our second hash function, where applicable, is (n % 7) + 1.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion?

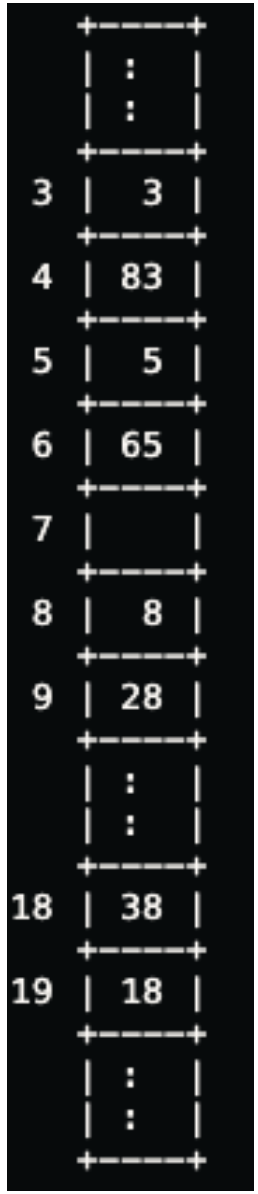


If we then search for 48, how many of the inserted keys will we look at?

On a search for 48, the algorithm will search the 8's bucket. Since the bucket has two elements, the search will look at two keys.

- In the following, say our hash function for n is n % 20.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion? If we then search for 48, how many of the inserted keys will we look at?
- Say we use linear probing for collision resolution instead. What is the answer to the above questions?

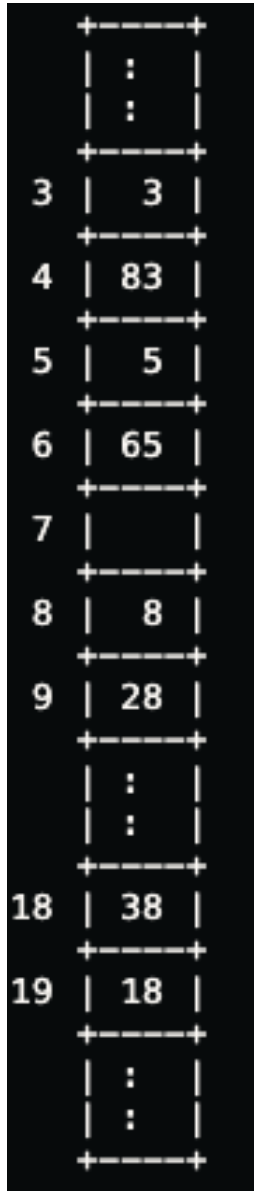
- In the following, say our hash function for n is n % 20.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion? If we then search for 48, how many of the inserted keys will we look at?
- Say we use linear probing for collision resolution instead. What is the answer to the above questions?



Collision at 28
Probing for 28:
1st probing: (28 % 20 + 1) % 20 = 9 no collision

Collision at 18
Probing for 18:
1st probing: (18 % 20 + 1) % 20 = 19 no collision

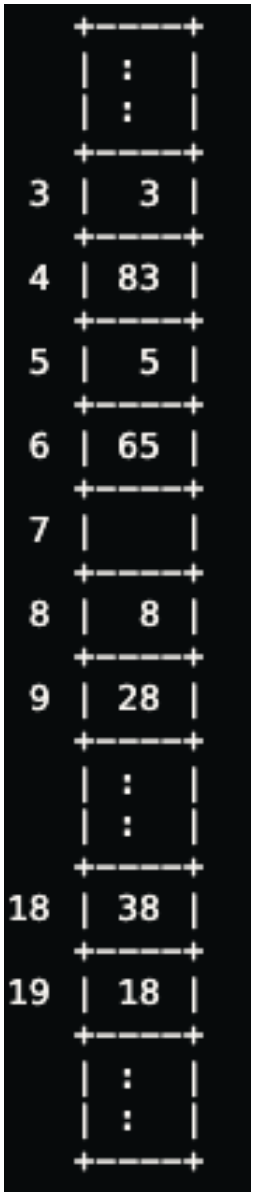
- In the following, say our hash function for n is n % 20.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion? If we then search for 48, how many of the inserted keys will we look at?
- Say we use linear probing for collision resolution instead. What is the answer to the above questions?



Collision at 65
Probing for 65:
1st probing: (65 % 20 + 1) % 20 = 6 no collision

Collision at 83
Probing for 83:
1st probing: (83 % 20 + 1) % 20 = 4 no collision

- In the following, say our hash function for n is n % 20 and our second hash function, where applicable, is (n % 7) + 1.
- Say we use separate chaining for collision resolution. Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- How will the hash table look after the final insertion? If we then search for 48, how many of the inserted keys will we look at?
- Say we use linear probing for collision resolution instead. What is the answer to the above questions?



On a search for 48, the algorithm will look at 8's bucket and move up until it either reaches 48 or it reaches an empty slot. In this case it looks at the keys in slots 8 and 9 before getting to the empty slot 10, so it looks at two of the inserted keys.

48 not found in 8's bucket

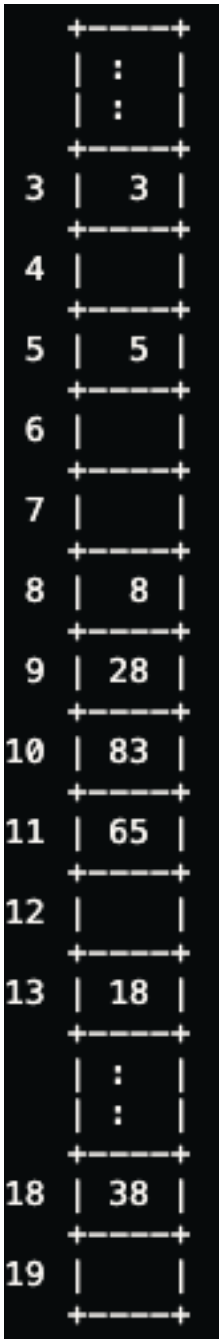
1st probing: (48 % 20 + 1) % 20 = 9, 48 not found in 9's bucket
2nd probing: (48 % 20 + 2) % 20 = 10, 10's bucket is empty

- In the following, say our hash function for n is n % 20 and our second hash function, where applicable, is (n % 7) + 1.
- Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- Say we use double hashing for collision resolution. How will the hash table look after the final insertion?

- In the following, say our hash function for n is n % 20 and our second hash function, where applicable, is (n % 7) + 1.
- Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- Say we use double hashing for collision resolution. How will the hash table look after the final insertion?

Collision at 28
Probing for 28:
1st probing: (28 % 20 + 1 * (28 % 7 + 1)) % 20 = 9 no collision

Collision at 18
Probing for 18:
1st probing: (18 % 20 + 1 * (18 % 7 + 1)) % 20 = (18 + 5) % 20 = 3 collision
2nd probing: (18 % 20 + 2 * (18 % 7 + 1)) % 20 = (18 + 10) % 20 = 8 collision
3rd probing: (18 % 20 + 3 * (18 % 7 + 1)) % 20 = (18 + 15) % 20 = 13 no collision



- In the following, say our hash function for n is n % 20 and our second hash function, where applicable, is (n % 7) + 1.
- Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- Say we use double hashing for collision resolution. How will the hash table look after the final insertion?

Collision at 65

Probing for 65:

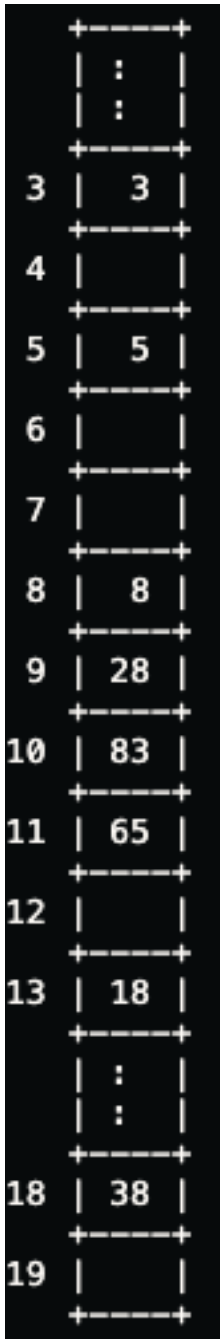
1st probing: (65 % 20 + 1 * (65 % 7 + 1)) % 20 = (5 + 3) % 20 = 8 collision

2nd probing: (65 % 20 + 2 * (65 % 7 + 1)) % 20 = (5 + 6) % 20 = 11 no collision

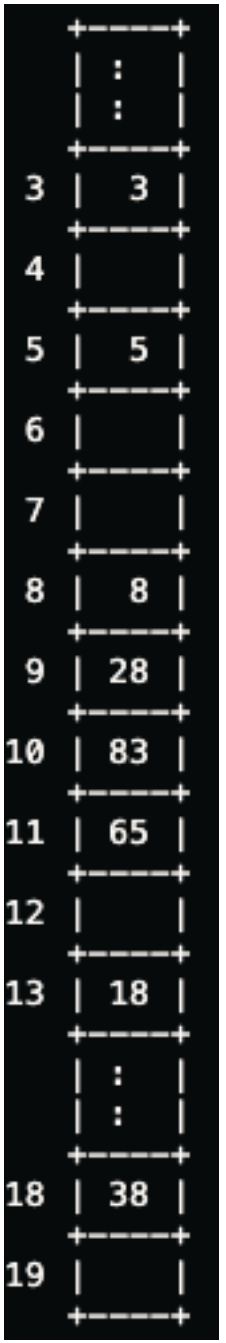
Collision at 83

Probing for 83:

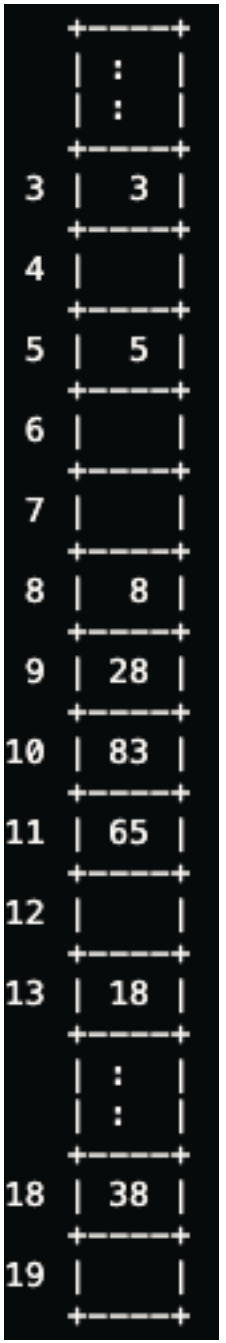
1st probing: (83 % 20 + 1 * (83 % 7 + 1)) % 20 = (3 + 7) % 20 = 10 no collision



- In the following, say our hash function for n is n % 20 and our second hash function, where applicable, is (n % 7) + 1.
- Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- Say we use double hashing for collision resolution. If we then search for 48, how many of the inserted keys will we look at?



- In the following, say our hash function for n is n % 20 and our second hash function, where applicable, is (n % 7) + 1.
- Beginning with an empty hash table, we insert the following.
- 8 38 3 5 28 18 65 83
- Say we use double hashing for collision resolution. Now what is the answer to the above questions? If we then search for 48, how many of the inserted keys will we look at?



On a search for 48, the algorithm will look at 8's bucket and move up until it either reaches 48 or it reaches an empty slot. In this case it looks at the key in slot 8 before reaching slot 15 and halting on this empty slot; so it sees one key only. Detailed steps are as follows:

48 not found in 8's bucket

1st probing: (48 % 20 + 1 * (48 % 7 + 1)) % 20 = (8 + 7) % 20 = 15, 15's bucket is empty